



Warsaw University of Technology

**Faculty of Mathematics
and Information Science**

Bachelor's diploma thesis

in the field of study Mathematics and Data Analysis

Machine Learning with Survival Outcomes

Agnieszka Kęska

student record book number 333064

thesis supervisor
Prof. Szymon Jaroszewicz

WARSAW 2026

Contents

1	Introduction	4
2	Fundamentals of Survival Analysis	5
	Basics of Survival Analysis	5
2.1	Basic Definitions	5
2.2	Censored data	7
2.2.1	Types of Censoring	7
2.2.2	Example	8
2.3	Statistical Estimators	10
3	Survival Machine Learning Models	14
3.1	Survival Tree	14
3.1.1	Decision Tree	14
3.2	Ensemble models	20
3.2.1	Bagging Survival Trees	20
3.2.2	Random Survival Forest	22
3.2.3	Gradient Boosting Machine	24
3.2.4	CoxBoost algorithm	27
3.3	Neural Networks	28
3.3.1	ANN COX	30
3.3.2	Cox-nnet	31
3.3.3	DeepSurv	31
4	Methods of Model Evaluation	32
4.1	Concordance Index	32
4.2	Brier Score	33

5 Experiments	34
5.1 Datasets	34
5.2 Conclusions	34

1 Introduction

Machine learning has become an important area of research in recent years. Machine learning methods are largely based on statistical and probabilistic models. These models can help in many fields to predict subjects of interest based on certain features. For instance, they can be used to predict which product a customer with specific characteristics is most likely to buy.

The aim of this thesis is to present the main machine learning models extended to survival data (that is, data where we want to predict the time to a certain event but some observations are censored) and to compare these models with each other and with basic statistical methods on different datasets to evaluate which performs best.

The structure of this thesis follows its objectives. The thesis contains of five chapters.

In the first chapter, the fundamentals of survival analysis will be explained, such as censoring, the fundamental functions that describe the target variable. Basic statistical models, such as the Kaplan-Meier estimator and the Cox model, will also be introduced.

In the second chapter, machine learning models such as survival trees, support vector machines (SVMs), and neural networks will be discussed. Their extensions to survival data will also be covered.

In the third chapter, the evaluation metrics for models applied to survival data will be defined.

In the fourth chapter, experiments on datasets will be performed. Various models will be applied to the data to assess their performance, and the results and conclusions will be presented in the final section.

This thesis is based on literature in machine learning and survival analysis.

2 Fundamentals of Survival Analysis

„Survival analysis is a collection of statistical procedures for data analysis where the outcome variable of interest is time until an event occurs.”
[3]

2.1 Basic Definitions

Let $T \geq 0$ be a random variable representing the survival (or event) time.

Survival Function

The survival function is the probability that an individual survives beyond time t :

$$S(t) = P(T > t), \quad 0 < t < \infty.$$

Cumulative Distribution Function (CDF)

The cumulative distribution function is defined as:

$$F(t) = P(T \leq t) = 1 - S(t).$$

Probability Density Function (PDF)

The probability density function is related to the survival function by:

$$f(t) = \lim_{\Delta t \rightarrow 0^+} \frac{P(t \leq T < t + \Delta t)}{\Delta t} = \frac{dF(t)}{dt} = -\frac{dS(t)}{dt}$$

where Δt denotes a time-interval.

Hazard Function

The hazard function describes the instantaneous risk of the event occurring at time t , given survival until time t :

$$h(t) = \lim_{\Delta t \rightarrow 0^+} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t}.$$

The hazard function can also be written as

$$h(t) = \frac{f(t)}{S(t)}.$$

because the following equalities from conditional probability hold:

$$\begin{aligned} h(t) &= \lim_{\Delta t \rightarrow 0^+} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t} = \lim_{\Delta t \rightarrow 0^+} \frac{1}{\Delta t} \frac{P(t \leq T < t + \Delta t)}{P(T \geq t)} = \\ &= \frac{1}{P(T \geq t)} \lim_{\Delta t \rightarrow 0^+} \frac{P(t \leq T < t + \Delta t)}{\Delta t} = \frac{f(t)}{S(t)}. \end{aligned}$$

Cumulative hazard function

$$H(t) = \int_0^t h(u) du.$$

Relationship between the Cumulative Hazard and Survival Function

The cumulative hazard function $H(t)$ is related to the survival function by:

$$S(t) = \exp(-H(t)).$$

because

$$H(t) = \int_0^t h(u) du = \int_0^t \frac{f(u)}{S(u)} du = \int_0^t \frac{-S'(u)}{S(u)} du = -\log(S(t))$$

[2][9]

2.2 Censored data

In many cases the exact time of the event is not known, as the observation may be censored. The dataset is then defined as follows: $y_i \geq 0$ denote the time when the event happened and $c_i \geq 0$ the time when the observation is censored. The censoring indicator is defined as:

$$\delta_i = \begin{cases} 1 & \text{if the event was observed } (y_i \leq c_i) \\ 0 & \text{if the i-th observation is censored } (y_i > c_i) \end{cases}$$

The Survival Dataset

A survival dataset is a matrix $D = (X, T, \delta)$ where

- X is the $n \times p$ feature matrix.
- $x_i = (x_{i1}, \dots, x_{ip})^T$ is a vector of p features for i-th observation.
- T is the vector of times observed $(t_1, \dots, t_n)$ where for $i = 1, \dots, n$:

$$t_i = \min(y_i, c_i) = \begin{cases} y_i & \text{if } \delta_i = 1 \\ c_i & \text{if } \delta_i = 0 \end{cases}$$

- $\delta = (\delta_1, \dots, \delta_n)$ is the vector of censoring indicators.

The Risk Set

The risk set is the set of observations which observed time is greater or equal the specific time t .

$$R(t) = \{i, t_i \geq t\}$$

2.2.1 Types of Censoring

The time of observation is:

- not censored when $y_i \in (l_i = t_i, r_i = t_i]$

- left-censored when $y_i \in [l_i = 0, r_i = t_i)$
- right-censored when $y_i \in (l_i = t_i, r_i = \infty)$
- interval-censored $y_i \in (l_i = m_i, r_i = s_i]$ where $m_i < s_i \leq t_i$

where l_i, r_i are random variables.[1]

2.2.2 Example

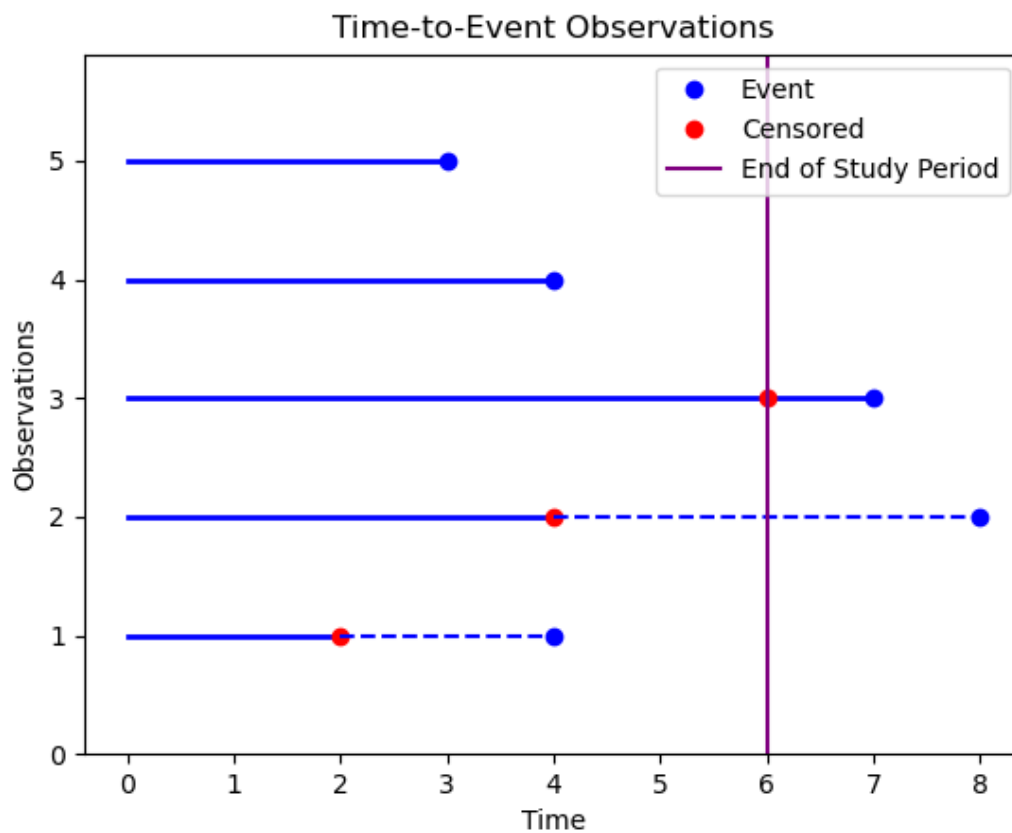


Figure 1:

Let us consider a study, whose aim is to analyse the time until young adults move out of their parents' house. The figure 1 presents the observed data. The study lasts 6 years. We observe that observations 4 and 5 are not censored, since the event (moving out of the family home) occurs before the end of the observation period (indicated by the purple line). Therefore, for these observations it is known what the exact time

of the event is.

The remaining observations (1, 2 and 3) are censored. The first observation corresponds to a drop-out of the study. This means that the individual was observed until time $t = 2$, but after that moment the observation was lost. We only know that the event did not occur before second year. In reality the person moved out in four-th year, but this information is unavailable because the individual was no longer tracked.

A similar situation occurs for the second observation, except that in this case the event occurs after the end of the study period

The third observation represents another type of censoring: the event did not occur during the observation period. We only know that it happens after six-th year, but the exact time is unknown.

In the mathematical notation this can be written as:

- for $i = 1, t_1 = \min(c_1 = 2, y_1 = 4) = 2, \delta_1 = 0$
- for $i = 2, t_2 = \min(c_2 = 4, y_2 = 8) = 4, \delta_2 = 0$
- for $i = 3, t_3 = \min(c_3 = 6, y_3 = 7) = 6, \delta_3 = 0$
- for $i = 4, t_4 = \min(c_4 = 6, y_4 = 4) = 4, \delta_4 = 1$
- for $i = 5, t_5 = \min(c_5 = 6, y_5 = 3) = 3, \delta_5 = 1$

2.3 Statistical Estimators

There are many statistical models used in survival analysis. These models can be divided into parametric, semi-parametric and non-parametric approaches. In this section, some of the most commonly used methods are described. In particular, the focus is on the non-parametric Kaplan-Meier estimator used to estimate the survival function and on the semi-parametric Cox proportional hazards model used to model the hazard function.

Let $D = (X, T, \delta)$ be a survival dataset as defined in Section 2.2.

Kaplan - Meier Estimator

Let $S(t)$ denote the survival function. The Kaplan-Meier estimator provides a non-parametric estimate of this function and is defined as:

$$\hat{S}(t) = \prod_{j:t_j \leq t} \left(1 - \frac{d_j}{n_j}\right)$$

where

- t_j denotes the j -th ordered event time. Let $t_1 < t_2 \dots < t_n$
- $n_j = |R(t_j)|$ is the number of observations at risk(that haven't been censored or experienced the event before t_j).
- $d_j = \sum_i \mathbf{1}_{\{t_i=t_j, \delta_i=1\}}$ is number of events in time t_j [2]

Example

Figures 2 and 3 show the Kaplan–Meier estimated survival function for the Rossi dataset [4], which concerns the recidivism of released prisoners over a follow-up period of 52 weeks. Figure 2 displays the overall survival function, while in the Figure 3 are presented two survival functions for two groups(individuals under 21 and others).

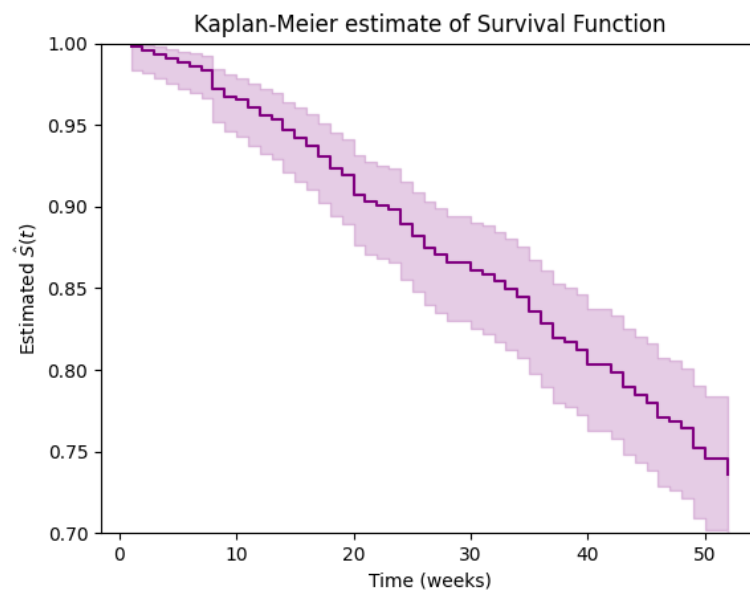


Figure 2:

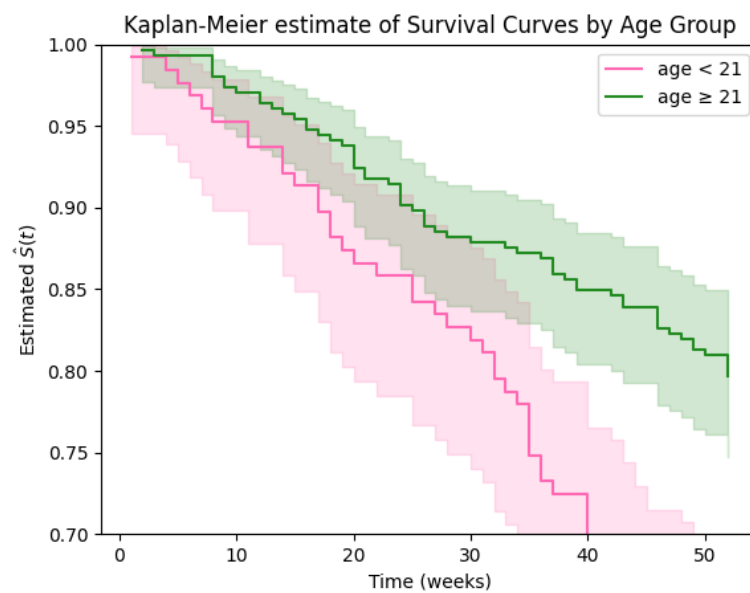


Figure 3:

Nelson-Aalen Estimator

The Nelson-Aalen Estimator is the statistic non-parametric method to estimate the cumulative hazard function

$$\hat{H}(t) = \sum_{j:t_j \leq t} \frac{d_j}{n_j}$$

- t_j denotes the j -th ordered event time. Let $t_1 < t_2 \dots < t_n$
- $n_j = |R(t_j)|$ is the number of observations at risk(that haven't been censored or event happend before t_j).
- $d_j = \sum_i \mathbf{1}_{\{t_i=t_j, \delta_i=1\}}$ is number of events in time t_j [2]

Cox Proportional Hazards Model

In the Cox model, hazard function for individual i is assumed to depend on their covariates x_i through:

$$h(t | x_i) = h_0(t) \exp(x_i^T \beta)$$

where h_0 is a baseline hazard function.

The Cox model is based on the proportional hazards assumption for any two individuals i and j :

$$\frac{h(t | x_i)}{h(t | x_j)} = \exp((x_i - x_j)^T \beta)$$

Survival function

$$\hat{S}(t|x_i) = S_0(t)^{\exp(x_i^T \beta)}$$

because

$$\hat{S}(t|x_i) = \exp(-\hat{H}(t|x_i)) = \exp(-H_0(t) \exp(x_i^T \beta)) = S_0(t)^{\exp(x_i^T \beta)}$$

$h_0(t)$, $H_0(t)$ and $S_0(t)$ denote the baseline hazard, baseline cumulative hazard and baseline survival function respectively.

to estimate $H_0(t)$ could be used Th Breslow's estimator[6, 1]:

$$\hat{H}_0(t) = \sum_{i:t_i \leq t} \frac{d_i}{\sum_{j \in R(t_i)} \exp(x_j^T \beta)}$$

Partial likelihood

The partial likelihood is based on comparing the relative risks of individuals within the risk set:

$$PL(\beta) = \prod_{i:\delta_i=1}^n l_i = \prod_{i=1}^n \frac{\exp(x_i^T \beta)}{\sum_{j \in R(t_i)} \exp(x_j^T \beta)}$$

where

$$l_i = \frac{\exp(x_i^T \beta)}{\sum_{j \in R(t_i)} \exp(x_j^T \beta)}$$

The coefficients β in Cox model are estimated by maximising the partial likelihood function:

$$\hat{\beta} = \arg \max_{\beta} PL(\beta)$$

3 Survival Machine Learning Models

In this chapter, machine learning models adapted to survival data are described. Each section first introduces the standard machine learning model, followed by its extension to the survival setting.

3.1 Survival Tree

3.1.1 Decision Tree

A decision tree is an algorithm used for classification or regression tasks. It consists of a root node, internal nodes, and leaf nodes. At each node, the data is split into two subsets according to a chosen criterion. The splitting criterion is usually based on a statistical test, which is chosen in such a way that it best separates the observations. Every tree has an ending, which are the leaves, where the observations finally fall. In the case of a classification problem, when an observation falls into a leaf, it is assigned to the class that dominates in this leaf. However, in the case of a regression problem, the output is the mean value of the observations in the leaf.

Survival Tree

A survival tree is an extension of the decision tree algorithm. The main idea is the same, however here we are interested in predicting the time to event for observations. The model is applied to datasets that contain survival data, including censored observations. To handle this type of data, the most commonly used splitting criterion is the log-rank test, which is widely used in survival analysis. The log-rank test is used to verify the null hypothesis that the survival functions of the compared groups are the same. In this type of decision tree we are interested in predict time to event so at the leaves it is used Kaplan-Meier estimator or Nelson-Aalen estimator which are computed separately for each leaf on the training data.

Algorithm

Input: Training Dataset $D = (X, T, \delta)_{i=1}^n$

Function BuildTree(D):

```
1. if stopping_criterion(D):  
    return Leaf(D)
```

```
best_split = None  
best_score = -infinity
```

```
2. for each variable j:  
    for each possible split s:  
        D1 = {  $x_i, t_i, \delta_i : x_i[j] \leq s$  }  
        D2 = {  $x_i, t_i, \delta_i : x_i[j] > s$  }  
        score = StatisticTest(D1, D2)  
        if score > best_score:  
            best_score = score  
            best_split = (j, s)
```

```
3. if best_split is None:  
    return Leaf(D)
```

```
 $j^* = \text{best\_split}[0]$   
 $s^* = \text{best\_split}[1]$   
D1 = {  $x_i, t_i, \delta_i : x_i[j^*] \leq s^*$  }  
D2 = {  $x_i, t_i, \delta_i : x_i[j^*] > s^*$  }  
#continue recursive building subtrees  
left_subtree = BuildTree(D1)  
right_subtree = BuildTree(D2)
```

```
return Tree
```

Splitting criteria

Distance-based criteria

One of the earliest approaches to splitting criteria for survival data was proposed by Gordon and Olshen (1985) [10]. The main idea was to construct a tree in which observations within each node are as homogeneous as possible with respect to event times. To quantify this homogeneity, they proposed using distances between survival distributions in candidate child nodes.

Let F_L and F_R denote the empirical distribution functions in the left and right nodes, respectively. The difference between the two distributions can be measured using the Wasserstein distance, defined as:

$$G = \int_0^1 |F_L^{-1}(u) - F_R^{-1}(u)| du$$

where $F_L^{-1}(u)$ and $F_R^{-1}(u)$ denote the quantile functions of the corresponding distributions, defined as:

$$F^{-1}(u) = \inf\{x : F(x) \geq u\}$$

Log-rank test based criteria

Let $t_1 < \dots < t_k$ be the ordered times and G_1 and G_2 denote two groups 1 and 2.

- $S_1(t)$ survival function in group G_1 and analogously $S_2(t)$ is the survival function for group G_2 .
- $D_{ik} = \sum_{j \in G_k} \mathbf{1}_{\{T_i=t_j, \delta_i=1\}}$ is the number of events in time T_i in group k , and $D_i = D_{i1} + D_{i2}$
- N_{ik} is number of observations at risk at i -th time in group k and $N_i = N_{i1} + N_{i2}$

- $E_{ik} = \frac{N_{ik}D_i}{N_i}$ expected events at i -th time in k -th group.
- $V_{ik} = E_{ik}(\frac{N_i - D_i}{N_i})(\frac{N_i - N_{ik}}{N_i - 1})$ is the variance of number of events in group k in i -th time.

where $k \in \{1, 2\}$

The log-rank statistic is defined as:

$$LR(G_1) = \frac{\sum_{k=1}^n (D_{k1} - E_{k1})}{\sqrt{\sum_{k=1}^n V_{k1}}}$$

and it is used for hypothesis testing:

$$\begin{cases} H_0 : S_1(t) = S_2(t) \text{ for each } t \\ H_1 : \text{not } H_0 \end{cases}$$

A large value of LR corresponds to a small p-value, leading to rejection of the null hypothesis H_0 for all t . In the survival tree algorithm, the split that maximises the log-rank statistic is selected at each node.

Likelihood-Based Splitting Criteria

An alternative approach to node splitting in survival trees is based on likelihood functions. Instead of comparing survival distributions directly, candidate splits are evaluated according to how well a probabilistic model fits the observations within the resulting child nodes. Below is presented one of several approaches.

Exponential likelihood approach

A common simplifying assumption is that survival times within each node follow an exponential distribution with a node-specific hazard parameter:

$$t_i \sim \text{Exp}(\lambda_t), \quad \lambda_t > 0$$

where t denotes a specific node in the tree. Under this assumption, the hazard function is constant over time:

$$h(t) = \lambda_t$$

Let G_t denote the set of observations belonging to node t .

For censored observations ($\delta_i = 0$), only the survival beyond time t_i is known, and the contribution to the likelihood is given by the survival function $S(T_i) = e^{-\lambda_t t_i}$. For uncensored observations ($\delta_i = 1$), the exact event time is observed and the contribution is given by the density function $f(t_i) = \lambda_t e^{-\lambda_t t_i}$. Combining both cases, the likelihood contribution of observation i can be written as:

$$l_i = f(t_i)^{\delta_i} S(t_i)^{1-\delta_i} = (\lambda_t e^{-\lambda_t t_i})^{\delta_i} (e^{-\lambda_t t_i})^{1-\delta_i} = \lambda_t^{\delta_i} e^{-\lambda_t t_i}$$

Thus, the likelihood for node t is:

$$L(\lambda_t) = \prod_{i \in G_t} l_i = \prod_{i \in G_t} \lambda_t^{\delta_i} e^{-\lambda_t t_i}$$

and the log-likelihood function:

$$l(\lambda_t) = \sum_{i \in G_t} \delta_i \log(\lambda_t) - \lambda_t \sum_{i \in G_t} t_i$$

Maximising the log-likelihood gives the maximum likelihood estimator:

$$\hat{\lambda}_t = \frac{\sum_{i \in G_t} \delta_i}{\sum_{i \in G_t} t_i}$$

Using the estimated hazard parameters, candidate splits can be evaluated by comparing the resulting likelihood values or deviance measures.

Pruning

Large decision tree models are prone to overfitting. To prevent this phenomenon, pruning methods are used. The main idea is to first construct a large tree and then iteratively remove branches in order to obtain an optimal subtree that balances model complexity and prediction error. There are two approaches presented below: the cost-complexity method and the split-complexity method.

Pruning algorithm

Input: Dataset D

1. Choose parameter $\alpha \geq 0$
2. Split the data into training and the validation **set**.
3. Build a large tree using the training set.
4. Iteratively remove branches from terminal nodes upward and calculate the error on the validation subset.
5. Choose the tree which maximizes split-complexity or minimizes cost-complexity.

Cost-complexity

$$R_\alpha(T) = R(T) + \alpha \cdot Complexity(T)$$

where $Complexity(T)$ is the number of leaf and $R(T)$ is the sum of errors in each leaf.

In the classification problems $R(t)$ is usually defined as the misclassification error, in the regression problem it is mainly sum of squares. However in the survival trees there could be various metrics used. $R(T)$ can be defined by the log-likelihood.

$$R(T) = \sum_{h \in L(T)} R(h), \quad \text{where} \quad R(h) = 2(LL_h(saturated) - LL_h(\tilde{\theta}_h)).$$

Where $LL_h(saturated)$ is the log-likelihood saturated model and $LL_h(\tilde{\theta}_h)$ is the max log-likelihood in the assumed model in node h .

Example

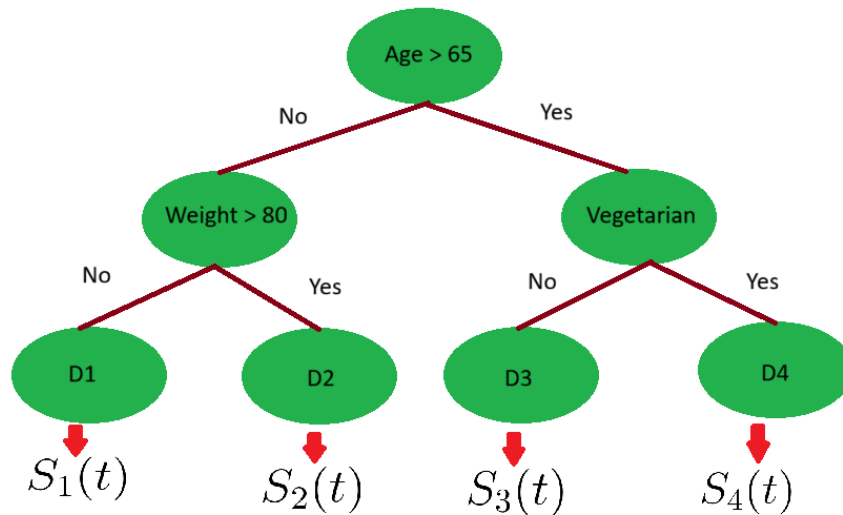


Figure 4:

Here D1,D2,D3,D4 are the four groups which arise from splits the initial dataset.

3.2 Ensemble models

3.2.1 Bagging Survival Trees

Bagging (Bootstrap Aggregating) is an ensemble learning method that combines multiple weak learners to improve predictive performance and reduce variance. Applied to survival trees, the procedure is as follows:

BST - Training Algorithm

Input: Dataset $D = \{(X, T, \delta)\}$, number of trees N .

Function $\text{BuildTree}(D)$:

1. **for** b **in** $(1, N)$:

 Create bootstrap sample D_b (random draw **with** replacement).

 Fit survival decision tree T_b

 on D_b dataset.

2. **return** $\text{BST} = \{T_1, \dots, T_N\}$

BST - Prediction Survival Function Algorithm

Input: observation x_i , BST.

1. **for** b **in** $(1, N)$:

 Drop an x_i to the T_b tree.

 It falls **in** the leaf L_{bi}

 So $\hat{S}_{bi} =$ kaplan maier function **in** leaf L_{bi}

2. $\hat{S}_i(t) = \sum_{b=1}^N w_b \hat{S}_{bi}(t)$

3. **Return** $\hat{S}_i$

where $w = (w_1, \dots, w_N)$ is the vector of weights such that $\sum_{i=1}^N w_i = 1$

Bagging Survival Trees is a model built on multiple survival trees. During the prediction stage, depending on the quantity of interest, the model can aggregate either the survival function estimated using the Kaplan-Meier estimator in each terminal node or the cumulative hazard function estimated using the Nelson-Aalen estimator in each terminal node. In the common case where

$$w = (w_1, \dots, w_N) = \left(\frac{1}{N}, \dots, \frac{1}{N} \right),$$

the estimates take the following form:

Estimate of the survival function:

$$\hat{S}_i(t) = \frac{1}{N} \sum_{b=1}^N \hat{S}_{bi}(t)$$

where $\hat{S}_{bi}(t)$ denotes the estimate of survival function for the i -th observation in the b -th survival tree.

Estimate of the cumulative hazard function:

$$\hat{H}_i(t) = \frac{1}{N} \sum_{b=1}^N \hat{H}_{bi}(t)$$

where $\hat{H}_{bi}(t)$ denotes the estimate of cumulative hazard function for the i -th observation in the b -th survival tree.

3.2.2 Random Survival Forest

Random Survival Forest (RSF) is an extension of the Bagging Survival Trees method. Similarly to the standard Random Forest algorithm, RSF is based on an ensemble of trees constructed using bootstrap sampling and random feature selection.

The Random Forest procedure can be summarized as follows:

1. Generate N bootstrap samples from the training dataset.
2. Train one tree on each bootstrap sample.
3. At each split, instead of considering all p features, randomly select $l < p$ features and determine the best splitting criterion among them.
4. Aggregate predictions from all trees.

Random Survival Forest differs from the standard Random Forest in the type of trees being constructed and the prediction mechanism. Instead of standard decision trees, RSF uses survival trees, where splitting criteria are adapted for censored survival data. The final prediction is ob-

tained by aggregating survival functions or cumulative hazard functions estimated in the terminal nodes of individual trees.

RSF - Training algorithm

Input Dataset $D = \{(X, T, \delta)\}$, number of selected features l , number of trees N .

1. **for** b **in** $(1, N)$:
 - i. Create bootstrap sample D_b
 - ii. Train survival tree T_b on D_b where at each split, randomly select $l < p$ features and determine the best split criterion.
2. **return** $RST = \{T_1, \dots, T_N\}$

RSF - Predicting Survival Function Algorithm

Input: observation x_i , RST.

1. **for** b **in** $(1, N)$:
 - Drop an x_i to the T_b tree.
 - It falls **in** the leaf L_{bi}
 - So $\hat{S}_{bi} =$ kaplan maier function **in** leaf L_{bi}
2. $\hat{S}_i(t) = \sum_{b=1}^N w_b \hat{S}_{bi}(t)$
3. Return $\hat{S}_i$

The final prediction in Random Survival Forest, analogously to Bagging Survival Trees, is obtained by aggregating the survival functions or cumulative hazard functions estimated in the terminal nodes of individual trees using the Kaplan-Meier and Nelson-Aalen estimators, respectively: In the common case where

$$w = (w_1, \dots, w_N) = \left(\frac{1}{N}, \dots, \frac{1}{N} \right),$$

the estimates take the following form:

Estimate of the survival function:

$$\hat{S}_i(t) = \frac{1}{N} \sum_{b=1}^N \hat{S}_{bi}(t)$$

where $\hat{S}_{bi}(t)$ denotes the estimate of survival function for the i -th observation in the b -th survival tree.

Estimate of the cumulative hazard function:

$$\hat{H}_i(t) = \frac{1}{N} \sum_{b=1}^N \hat{H}_{bi}(t)$$

where $\hat{H}_{bi}(t)$ denotes the estimate of cumulative hazard function for the i -th observation in the b -th survival tree.

3.2.3 Gradient Boosting Machine

Gradient Boosting Machine (GBM) is also an ensemble method, but it differs fundamentally from bagging. Rather than building N independent trees in parallel, GBM fits trees sequentially: each new tree is trained to correct the residual errors of the current ensemble.

The standard gradient boosting machine for regression is built as follows:

GBM - Training Algorithm

Input: Dataset D , differentiable loss function, step size ν

1. First guess g_0
2. For m in $(1, N)$:
$$r_{im} = -\frac{\partial L(y_i, \hat{g}_{m-1}(X_i))}{\partial \hat{g}_{m-1}(X_i)}, i \in \{i : x_i \in D\}$$
$$\hat{g}_m \leftarrow \text{fit machine learning model } m \text{ to } (X_i, r_m)$$
$$\hat{g}_m \leftarrow \hat{g}_{m-1} + \nu \hat{g}_m$$
2. return $\hat{g}_N$

GBM - Predicting Algorithm

Input: GBM, step size ν , observation x_i

1. $\hat{y} = 0$
2. For m in $(1, N)$:
$$\hat{y}_m \leftarrow \hat{y}_{m-1} + \nu \hat{y}_m(x_i)$$
3. return $\hat{y}_N$

For regression problems, the squared error loss is commonly used:

$$L(y, \hat{y}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

GBM Cox

In the Cox proportional hazards setting, the weak learner is defined as

$$g_m(X) = X^T \beta_m,$$

where β_m is the vector of coefficients estimated from the Cox proportional hazards model. Consequently, the loss function is defined as the negative Cox partial log-likelihood.

Partial likelihood

$$PL(\beta) = \prod_{i:\delta_i=1}^n \frac{\exp(x_i^T \beta)}{\sum_{j \in R(t_i)} \exp(x_j^T \beta)}$$

Negative Cox Partial Log Likelihood

$$-l^{PH}(\beta) = - \sum_{i=1}^n \delta_i [x_i^T \beta - \log(\sum_{j \in R(t_i)} \exp(x_j^T \beta))]$$

Then the residuals looks like this:

$$r_{im} = \delta_i - \sum_{j=1}^n \delta_j \frac{\mathbf{1}_{t_i \geq t_j} \exp(g_{m-1}(X_i))}{\sum_{k \in R_{t_j}} \exp(g_{m-1}(X_k))}$$

where

$$g_{m-1}(X_i) = X_i \beta_{m-1}$$

CoxBoost

The next boosting method is CoxBoost. It is not a standard GBM approach, since instead of constructing complex base learners, it performs variable-wise updates of the coefficients in the Cox proportional hazards model using the boosting principle.

Let us define the following sets:

- $I = (1, \dots, p)$ is a set of features
- O is a set of optional attributes the first set $O_0 = I$
- $M = I - O$ is a set of mandatory attributes
- $I_{mk} = M \cup \{k\}, k \in O$
- λ is chosen to the regularization problem
- $P \in \mathbb{R}^{p \times p}$ is the penalty matrix where $P_{ii} = 0$ if $i \in M$

The penalty Cox partial log-likelihood function is defined as:

$$l_{pen}(\gamma_{mk}) = \sum_{i=1}^n \delta_i [x_i \beta_{m-1} + x_{i,I_{mk}} \gamma_{mk} - \log(\sum_{j=1}^n \mathbf{1}_{(t_j \leq t_i)} \exp(x_i \beta_{m-1} + x_{i,I_{mk}} \gamma_{mk}))] - \lambda \gamma_{mk}^T P_{mk} \gamma_{mk}$$

l_{pen} has the maximum value for:

$$\gamma_{mk}^* = H^{-1}(\hat{\gamma}_{(m-1)k}) U(\hat{\gamma}_{(m-1)k})$$

where $U(\gamma) = \frac{\partial l}{\partial \gamma}$ and $H(\gamma) = \frac{\partial^2 l}{\partial^2 \gamma}$ l - unpenalized cox partial log-likelihood.

The $\argmax_{\gamma}(l_{pen})$ is obtained by Taylor approximation:

$$l(\beta + \gamma) \approx l(\beta) + U(\beta)^T \gamma - \frac{1}{2} \gamma^T H(\beta) \gamma$$

$$\frac{\partial l}{\partial \gamma}(\beta + \gamma) \approx U(\beta) - H(\beta) \gamma$$

to maximize this function the equal to zero is constructed:

$$U(\beta) - H(\beta) \gamma^* = 0$$

$$U(\beta) = H(\beta) \gamma^*$$

$$\gamma^* = H^{-1}(\beta) U(\beta)$$

3.2.4 CoxBoost algorithm

Input Dataset D

Function CoxBoost(D):

1. Initialize $\hat{\beta}_0 = 0$
2. For m in (1,N):
 - i. For k in O:
 1. Create candidate set $I_{mk} = M \cup \{k\}$
 2. Calculate $l_{pen}(\gamma_{mk})$
 3. Calculate $\gamma_{mk} = H^{-1}(\beta_m) U(\beta_m)$

- ii . $k^* = \arg \max_k l_{pen}(\hat{\gamma}_{mk})$
 - iii . $\hat{B}_m = \hat{B}_{m-1} + \hat{\gamma}_{(m-1)k^*}, k^* \in I_{mk}$
3. **return** $\hat{\beta}_N$

3.3 Neural Networks

Neural networks are flexible predictive models that form the basis of many deep learning methods. Their structure is loosely inspired by biological neural systems, where neurons exchange information. A neural network consists of an input layer, one or more hidden layers, and an output layer. Each hidden layer is composed of multiple perceptrons, which transform the input through weighted linear combinations followed by activation functions.

Perceptron

A perceptron is the basic computational unit of a neural network. It computes a weighted linear combination of inputs followed by an activation function: suppose we have input data X then the prediction is $Y = f(w_0 + \sum_{i=1}^m w_i X_i)$ where f is an activation function.

popular activation functions:

- ReLU function $f(z) = \max(0, z)$
- sigmoid function $f(z) = \frac{1}{1+e^{-\beta z}}$
- hyperbolic tangent $f(z) = \frac{e^{\beta z} - e^{-\beta z}}{e^{\beta z} + e^{-\beta z}}$
- identity $f(z) = z$

A neural network with a single hidden layer can be described by the following three equations:

$$Z_m = \sigma(w_{0m} + w_m^\top x_i), \quad m = 1, \dots, M$$

$$T = B_{0k} + B_k^T Z, \quad k = 1, \dots, K$$

$$g_k(x_i) = f_k(T)$$

where Z_m is the activation of the m -th hidden neuron, M denotes number of hidden units and K denotes the number of outputs of the network.

$$g_k(x_i) = \sigma_0(\beta_{k0} + \sum_{h=1}^H (\beta_{kh} \sigma_h(\beta_{h0} + \sum_{m=1}^M \beta_{hm} x_{i;m})))$$

output layer

$$T = \beta_0 + \beta^\top Z,$$

prediction function

$$g(x_i) = \phi(T),$$

where $\phi(\cdot)$ is the output activation function.

Backpropagation

Backpropagation is a method for optimizing neural network parameters by minimizing a loss function, where gradients are computed using the chain rule and errors are propagated backward through the network.

NN - training algorithm

Input: Dataset $D = (X, Y)$

Function TrainNN(D):

1. Initialize $\theta \leftarrow$ random weights

2. For j in $(1, J)$:

i. Pass the **input** data through the network using the current parameter values $= \theta^{(j-1)}$

ii. Compute predicted outputs:

$$\hat{Y} = g(X|\theta^{(j-1)})$$

iii. Calculate $L(Y|D)$ loss function

iv. Backpropagation – Compute gradients $L(\theta)$ using chain rule

iv. Update

$$\theta \leftarrow \theta - \nu \nabla_{\theta} L(\theta)$$

3. Return θ

Survival Artificial Neural Networks

3.3.1 ANN COX

The ANN-Cox model is an extension of the Cox proportional hazards model by incorporating a neural network structure. Similar to the Cox model, it assumes the proportional hazards data. It assumes the proportional hazard data. The architecture consists of a single hidden layer, and the network output represents an estimated risk score. The predicted risk can be expressed as:

$$\hat{g}(X) = f(Xw)$$

where the training algorithm is used the partial likelihood as the loss function:

$$L(\hat{g}, \theta) = \prod_{i=1}^n \frac{\exp(\sum_{m=1}^M \beta_m \hat{g}_m(X))}{\sum_{j \in R_{t_j}} \exp(\sum_{m=1}^M \beta_m \hat{g}_m(X))}$$

$\theta = (w, \beta)$ and $\hat{g}_m(X) = \text{sigmoid}(X\beta)$ is m-th hidden unit output of activation function so the final output is $\hat{g}(X) = f(\sum_{m=1}^M \hat{\beta}_m \hat{g}_m(X))$, where f is an activation function.

The hazard function takes the following form:

$$h(t|X_i, \theta) = h_0(t) \exp(g(\hat{X}))$$

3.3.2 Cox-nnet

Cox nnet is an extension of an artificial neural network it also consists of only one hidden layer however in this model the Cox partial log-likelihood with a regularization term is here considered

$$L(\hat{g}, \theta) = \sum_{i=1}^n \delta_i [\hat{g}(x_i) - \log(\sum_{j \in R_{t_i}} \exp(\hat{g}(x_j)))] + \lambda(\|w\|_2 + \|\beta\|_2)$$

here $\theta = (w, \beta)$ where denotes the model parameters, w represents the weights of the hidden layer, and β is the output layer coefficient vector. The nonlinear risk function is defined as:

$$\hat{g}(x_i) = \tanh(wx_i + b)^T$$

where b denotes the bias term and $\tanh()$ is the hyperbolic tangent activation function.

3.3.3 DeepSurv

DeepSurv is an extension of the previously discussed neural network-based survival models to a deep neural network architecture with multiple hidden layers. Unlike Cox-nnet, DeepSurv can model more complex nonlinear relationships between covariates and the hazard function due to its deeper structure.

The model is trained by minimizing the negative Cox partial log-likelihood with L_2 regularization. Additionally, the loss function is scaled by the number of uncensored observations:

$$L(\hat{g}, \theta) = -\frac{\sum_{i=1}^n \delta_i \left[\hat{g}(x_i) - \log \left(\sum_{j \in R_{t_i}} \exp(\hat{g}(x_j)) \right) \right]}{\sum_{i=1}^n \mathbf{1}(\delta_i = 1)} + \lambda \|\theta\|_2^2$$

where θ denotes all trainable parameters of the neural network, including weights and biases, δ_i is the censoring indicator, and R_{t_i} represents the risk set at time t_i . The function $\hat{g}(x_i)$ corresponds to the estimated nonlinear log-risk function produced by the neural network.

4 Methods of Model Evaluation

In this section there are presented methods used to evaluate and compare machine learning models for survival analysis.

4.1 Concordance Index

In survival analysis, the concordance index is commonly used to evaluate predictive model performance. Its purpose is to assess how well a model correctly ranks individuals according to their predicted risk. In other words, it measures the proportion of comparable observation pairs for which the predicted risk ordering agrees with the observed survival ordering.

Not all pairs of observations can be compared due to censoring. A pair of observations is considered comparable if one of the following conditions is satisfied:

Let (i, j) be the pair of observation from a dataset $D(X, T, \delta)$

The (i, j) is comparable if:

- $\delta_i = 1$ and $\delta_j = 1$

or

- $t_i < t_j$, where $\delta_i = 1$ and $\delta_j = 0$

C-index

Assume that $y = (y_1, \dots, y_n)$ is the vector of predicted risk scores, $t = (t_1, \dots, t_n)$ the corresponding survival times, and $\delta = (\delta_1, \dots, \delta_n)$ the censoring indicators. For a non-negative weighting function W , the survival concordance index is defined as:

$$C(y, t, \delta \mid \tau) = \frac{\sum_{i \neq j} W(t_i) \mathbb{I}(t_i < t_j, y_i > y_j, t_i < \tau) \delta_i}{\sum_{i \neq j} W(t_i) \mathbb{I}(t_i < t_j, t_i < \tau) \delta_i},$$

where τ is the end of the study period.

This index can take values in the $[0, 1]$ interval. A value of 1 indicates perfect concordance between predicted and observed risk ordering, whereas a value of 0.5 corresponds to random prediction. Values below 0.5 indi-

cate performance worse than random ranking.

4.2 Brier Score

The Brier score is another commonly used metric in survival analysis and measures the prediction error of a model at a specific time point. It is defined as the mean squared difference between the true survival status and the estimated survival probability at time t . Lower Brier score values indicate better predictive performance, whereas higher values suggest poorer model accuracy.

For censored survival data the Brier score is defined as:

$$BS(t) = \frac{1}{N} \sum_{i=1}^N \left(\frac{\left(0 - \hat{S}(t, x_i)\right)^2 \mathbf{1}_{(t_i \leq t, \delta_i=1)}}{\hat{G}(T_i^-)} + \frac{\left(1 - \hat{S}(t, x_i)\right)^2 \mathbf{1}_{(t_i > t)}}{\hat{G}(t)} \right),$$

where $\hat{S}(t, x_i)$ denotes the predicted survival probability for observation i at time t , and $\hat{G}(t) = P(C > t)$ denotes the Kaplan-Meier estimator of the censoring survival function.

5 Experiments

5.1 Datasets

5.2 Conclusions

References

- [1] Sonabend. R, Bender. A. (2025). Machine Learning in Survival Analysis. [Book](#)
- [2] Christiana Kartsonaki, Survival analysis, Diagnostic Histopathology, Volume 22, Issue 7, 2016 [Article](#)
- [3] Clark TG, Bradburn MJ, Love SB, Altman DG. Survival analysis part I: basic concepts and first analyses. Br J Cancer. 2003. [Article](#)
- [4] Rossi, P.H., R.A. Berk, and K.J. Lenihan (1980). Money, Work, and Crime: Some Experimental Results. New York: Academic Press.
- [5] Davidson-Pilon, C. lifelines, survival analysis in Python [Computer software]. [Article](#)
- [6] Breslow, N. 1972. "Discussion following 'Regression models and life tables' by D. R. Cox." Journal of the Royal Statistical Society: Series B (Statistical Methodology) 34 (2): 187–220.
- [7] Nichols JA, Herbert Chan HW, Baker MAB. Machine learning: applications of artificial intelligence to imaging and diagnosis. Biophys Rev. 2019 [Article](#)
- [8] Sonabend, Raphael Edward Benjamin. A theoretical and methodological framework for machine learning in survival analysis: Enabling transparent and accessible predictive modelling on right-censored time-to-event data. Diss. UCL (University College London), 2021. [Article](#)
- [9] Jenkins, Stephen P. "Survival analysis." Unpublished manuscript, Institute for Social and Economic Research, University of Essex, Colchester, UK 42.54-56 (2005): 1. [Article](#)
- [10] Gordon, Louis, and Richard A. Olshen. "Tree-structured survival analysis." Cancer treatment reports 69.10 (1985): 1065-1069.